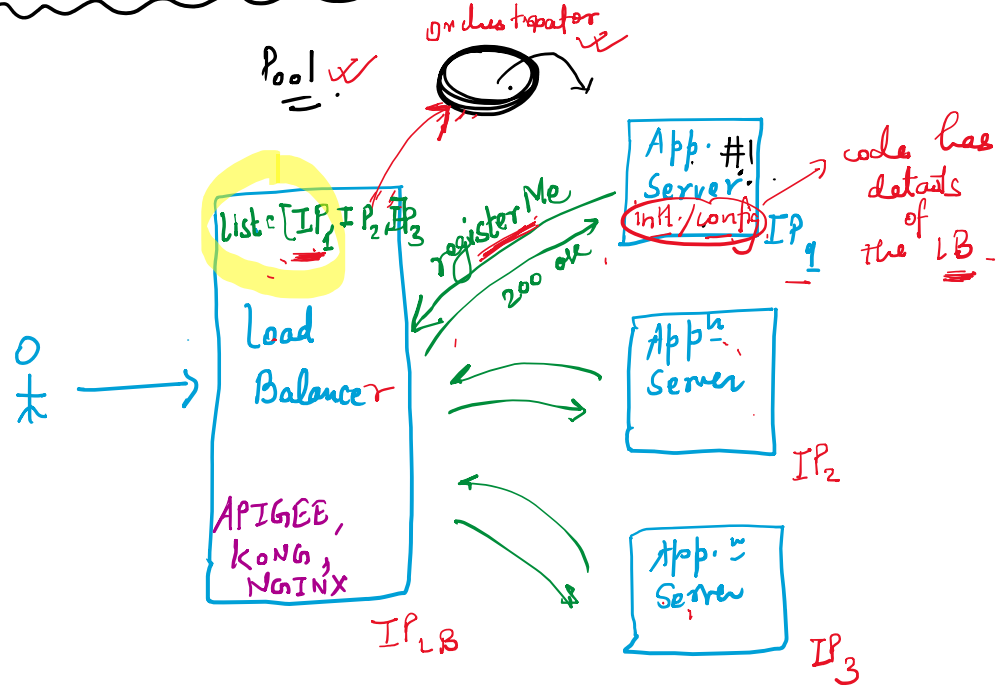
Wednesday, September 16, 2026 8:39 PM

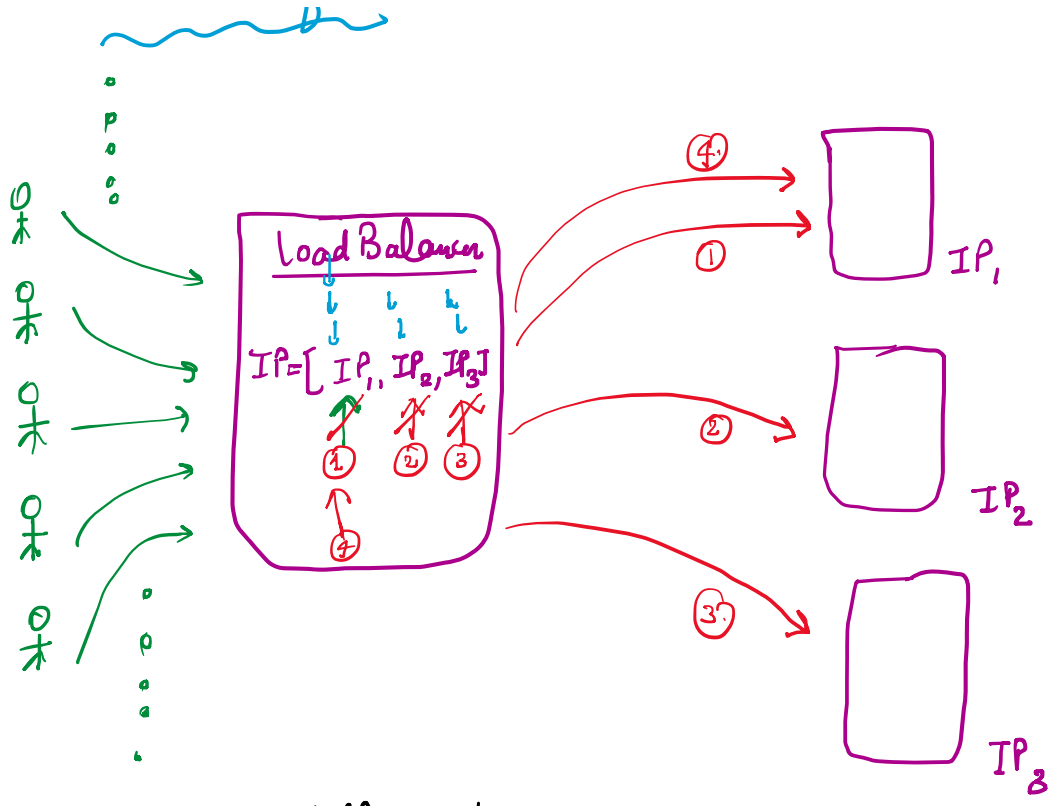
Working of load Balancer (LB)

(1.) How LB gets to know about app^n. server?



When an app^n server starts, it (AS) registers w/ the LB.

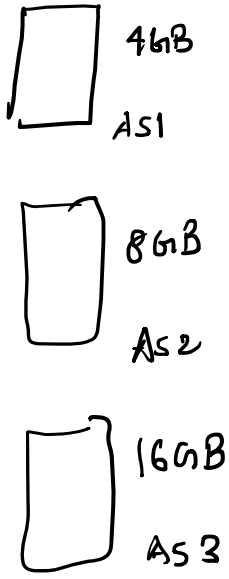
(2.) How the LB decides which server to fwd req. to?



Ans/Soln :- Use diff. Algo :=

1. Round Robin
   - Don't think much
   - most Popular

2. Weighted Round Robin
   - Not all AS have same capacity



AS1 : AS2 : AS3 ≡ 4 : 8 : 16

(cap. to handle req.)

⇒ Out of 28 req/sec, it makes sense

to send 4 → AS1

8 → AS2

16 → AS3

Similarly 56 req/sec $\frac{4}{28} \times 56 : \frac{8}{28} \times 56 : \frac{16}{28} \times 56$

Note :- Not necessarily RAM is the only deciding factor for the ratio.

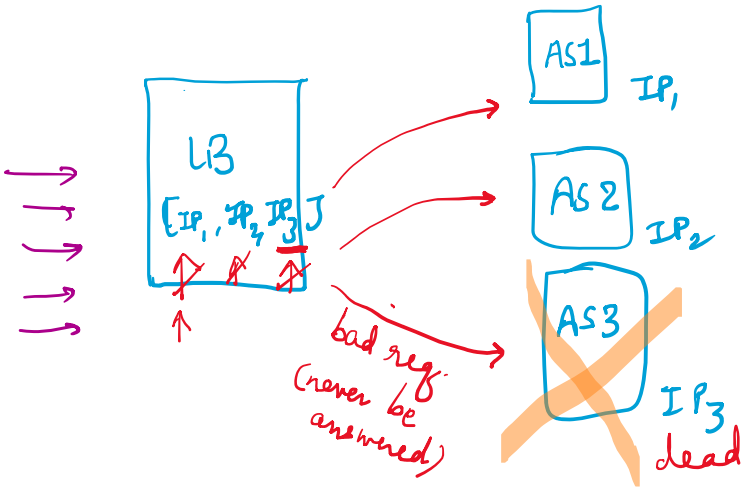
** Note [The routing algo (RR, weighted-RR, etc.) MUST be as light-weight as possible.]

↳ shouldn't behave as bottle-neck in your entire architecture. ≡.

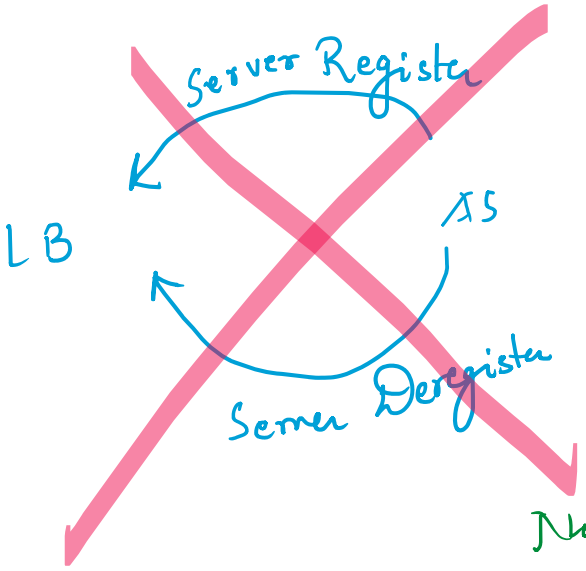
3. What if a server (eg: AS) dies ?

(3.) What if a server (AS) dies?

↳ How will LB get to know a server has died?



∴ LB must know if an already registered AP is no longer available ∵



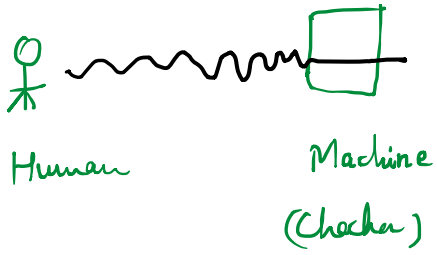
Not possible because servers generally don't know when they would die/crash.

↳ Soln / Analogy

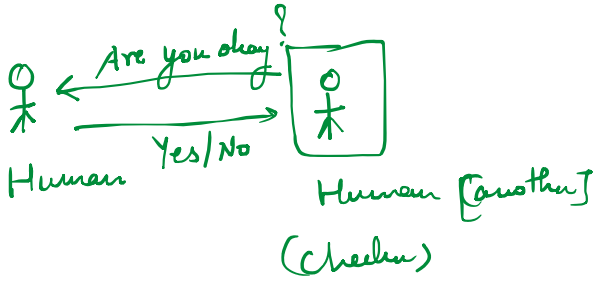
... check if a human is alive &

How do we check if a human is alive ?

1. Hospital
   - ↳ HeartBeat



2. Normal
   - ↳ Health check



Load Balancer

- ↳ HeartBeat
  - ↳ every server has to send a 'hello'

↳ every server has to send a hello to LB every 't' secs.

↳ if LB doesn't get a hello from a server for consecutive 'n' times, ie for consecutive (n*t) secs.

then LB assumes that server is dead.

→ HealthCheck

→ LB will ask server every 't' sec. if they are available (ping ↓ pong)

→ If the server doesn't respond for consecutive 'n' times ie, (n*t) consecutive duration then LB assumes the server to be dead

10:06 pm

Consistent Hashing.

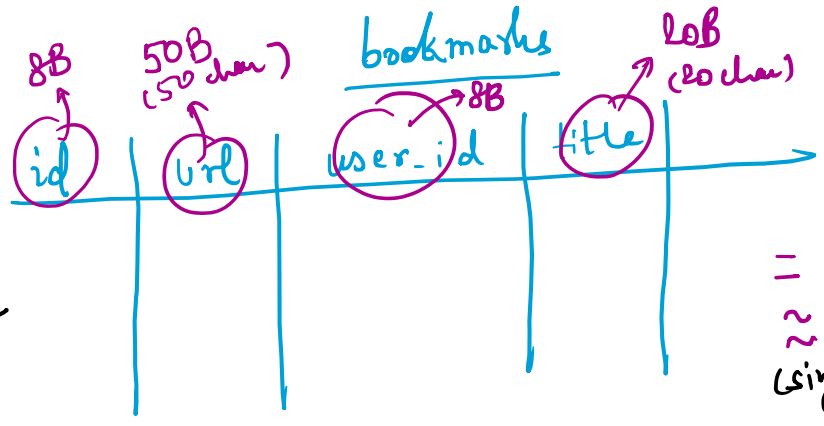
Where is our data stored / fetched from ?

↳ DB

Eg. → delicious

Assume [#users → 50 M (million)
avg. bookmarks/day/user → 0.25
(one bookmark per user every 4 days)

⇒ # bookmarks/day ≈ 12.5 million



Assume

= 96B
≈ 100B
(single bookmark entry in DB)

$\Rightarrow 12.5 * 10^6 * 100$ bytes / day

$= 1.25 * 10^9$ bytes / day

≈ 1.25 GB / day

$\approx$ 1 GB / day

∴ You need to store 1GB of new data per day.

2^{10} bytes = 1 kB $\Rightarrow$ 1024 bytes = 1kB

Approx. 1000 bytes $\approx$ 1 kB

**

2^{10} bytes = 1kB $\Rightarrow$ 10^3 bytes $\approx$ 1kB

2^{10} kB = 1MB $\Rightarrow$ 10^3 kB $\approx$ 1 MB

2^{10} MB = 1GB $\Rightarrow$ 10^3 MB $\approx$ 1GB

2^{10} GB = 1TB $\Rightarrow$ 10^3 GB $\approx$ 1TB

2^{10} TB = 1PB $\Rightarrow$ 10^3 TB $\approx$ 1PB

Now · 1 GB / day

⇒ 1 year ⇒ 365 * 1GB
= 365 GB
≈ 400 GB

⇒ 5 years ⇒ 5 * 400 GB
≈ 2 TB data.

If size of disk was 40GB

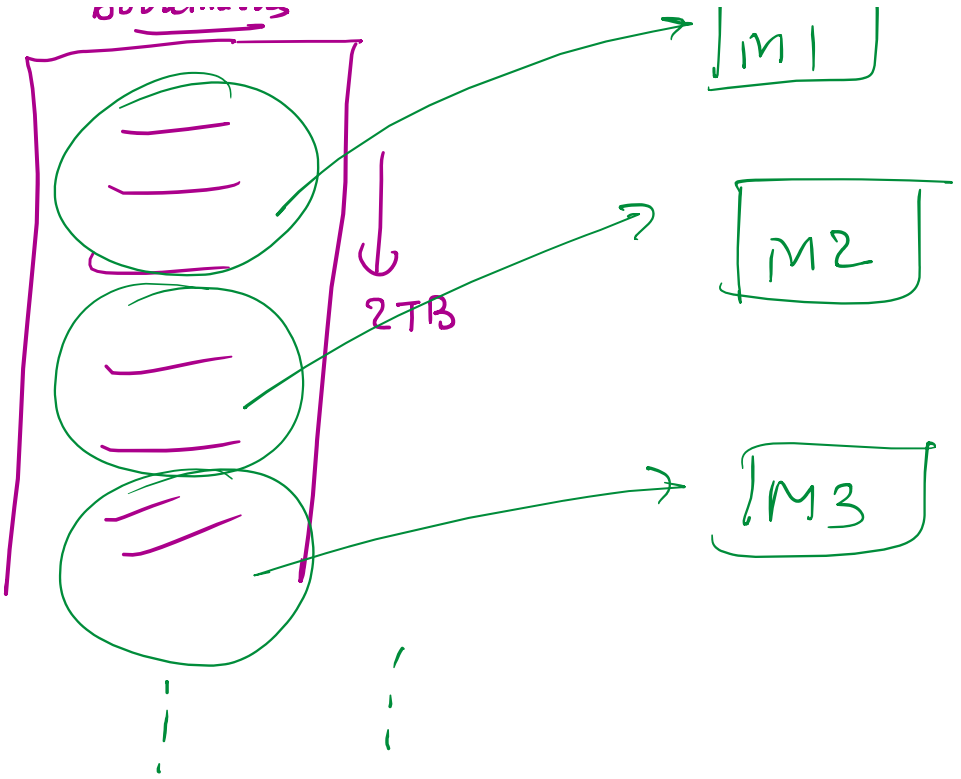
⇒ new disk (DB) (m/c) every 40 days.

⇒ all data of delicious can't fit in 1 machine.

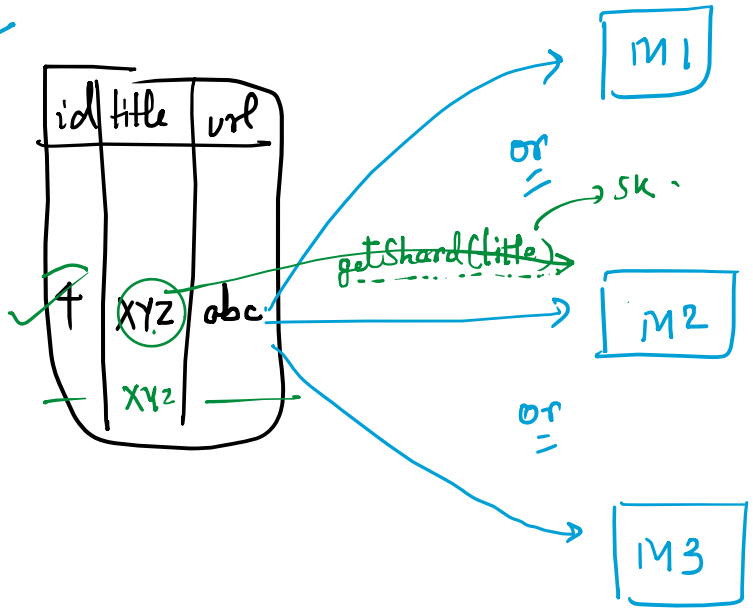
∴ I will have to divide data amongst multiple machines.

shards

bookmarks



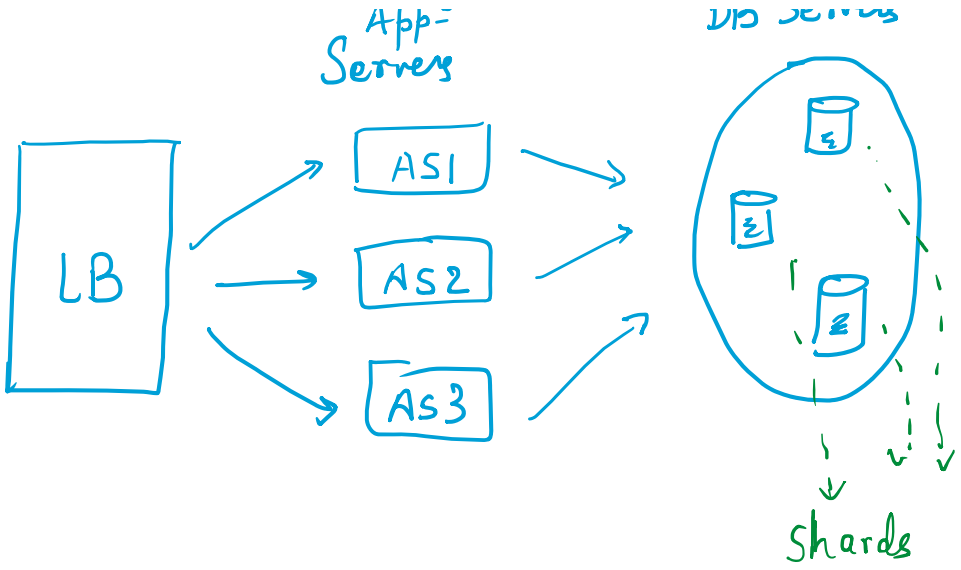
PS



SHARDING ⇒ dividing the data of a DB table into diff. (physical) machines

App^n Servers

DB Servers



19/9/26

(*) How to shard ?

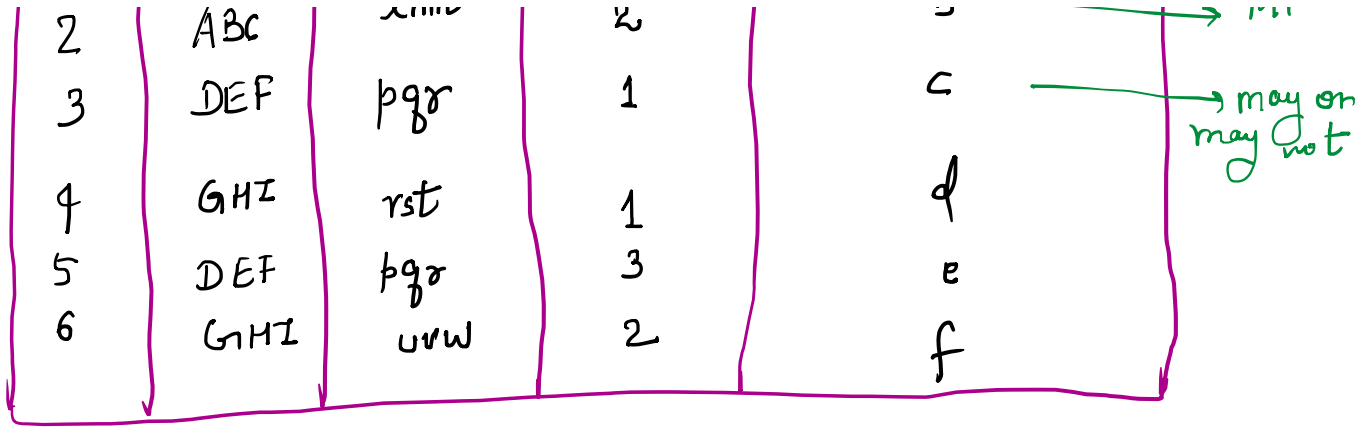
(1.) Choose a sharding key **

an attribute basis whose value you decide to divide the data.

bookmarks

created-by.

id	title	url	user-id	timestamp.	Shard no.
1	ABC	uvw	1	a	→ M1
2	ABC	lmn	2	b	→ M1
3	DEF	pqr	1	c	→ ...



Eg. title

↳ all rows (records) w/ the same title (the same value of the attribute)

will go to the same shard.

Consistent Hashing: getShard (title) → Shard #

- What is the correct sk (2.)
- ~~Meaning~~ of sk (1.)
- If sk has been chosen, how to decide which sk value goes to which m/c. (3.) → Consistent Hashing

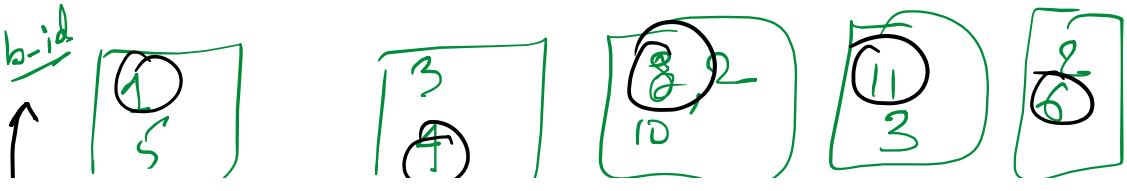
⊛ user_id (sk)

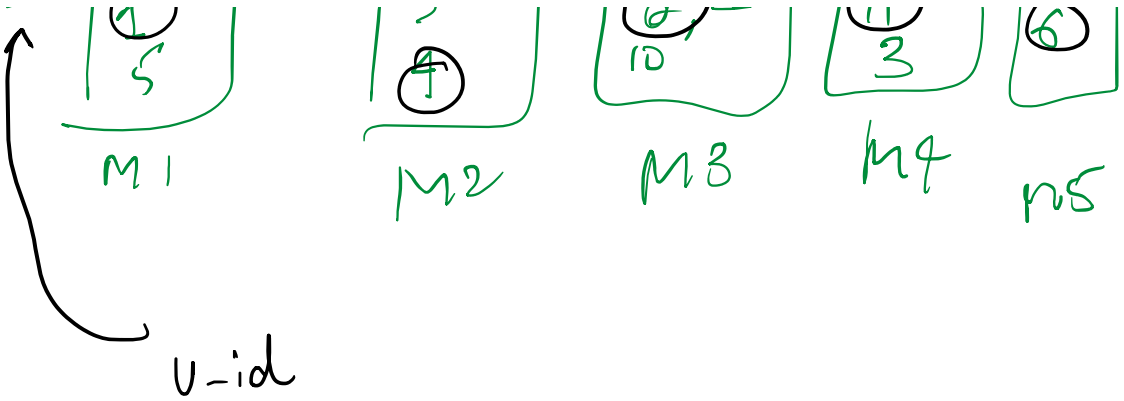
↳ All data w/ same value of user_id
— will go to the same shard.

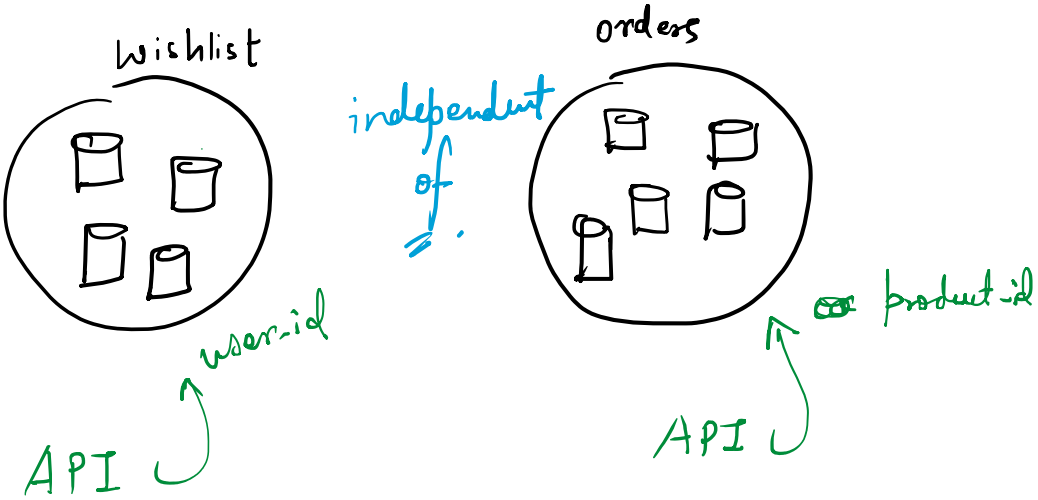
Note :- NOT every sk will necessarily have a diff shard.

⊛ What will be a good sk for del.icio.ous?
= (How to choose a good sk)

sk-candidate	getBookmarks(u-id)	createBookmark(u-id, title, ----) (single Bookmark creation)
id	ALL	1
title	ALL	1
url	ALL	1
user_id	1	1
timestamp	ALL	1

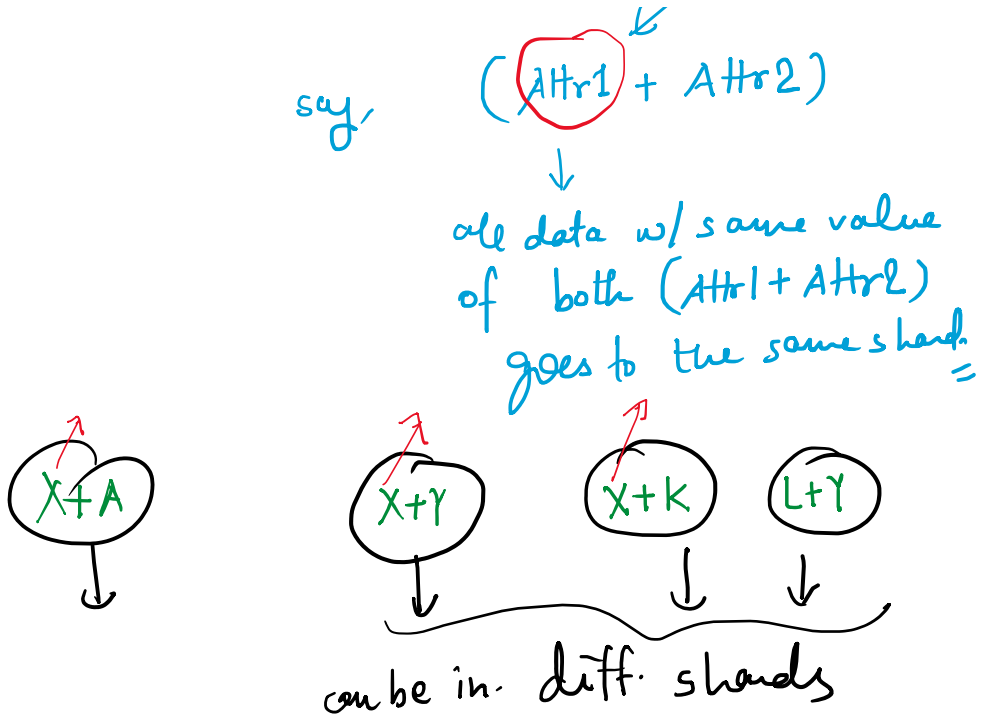






Note :- SK is not chosen as a composite.

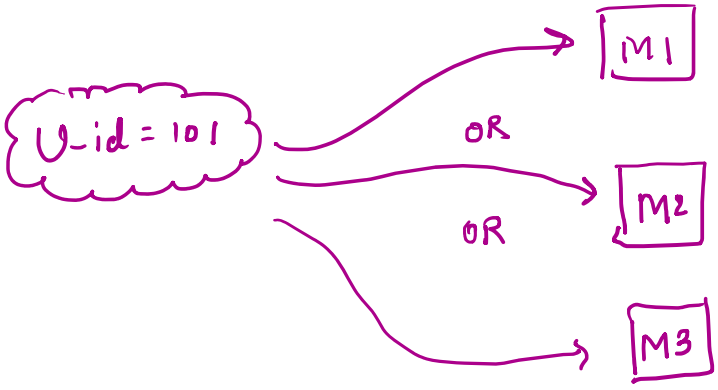
(Attr1 + Attr2)

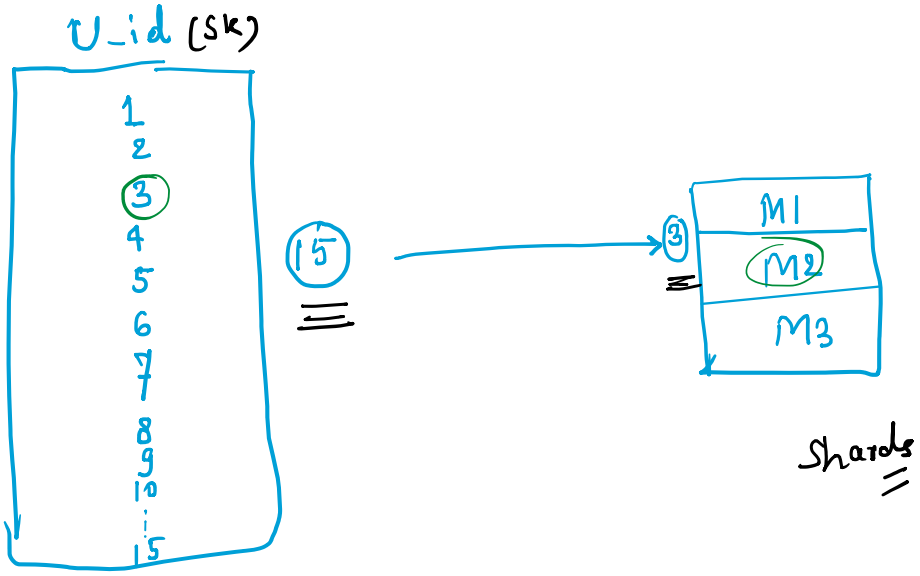


Caveat :- If damn sure that all APIs are purely based on that composite SK. then Ok to have it.

⊛ All rows with same value of 'X' (SK) to be going to the same m/c (shard)

what m/c

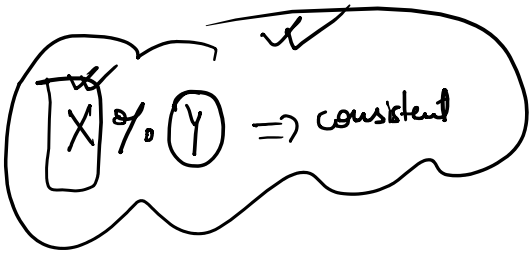




The sol^{n} of routing has to be consistently accurate

↳ hence the name

Consistent Hashing



Properties of the ideal sol^{n} :-

1.) Fast (so that the routing itself doesn't become bottleneck of overall perf.)

2.) Uniform dist. of data

→ ideally :- no rearrangement

2.) Uniform [illegible]

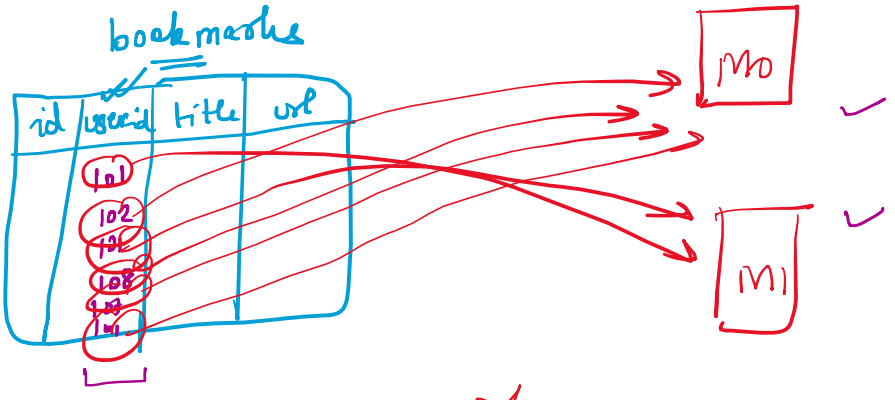
3.) Add^{n} / Removal of shard should be fast

- ideally :- no rearrangement at all
- practically :- as less as possible.

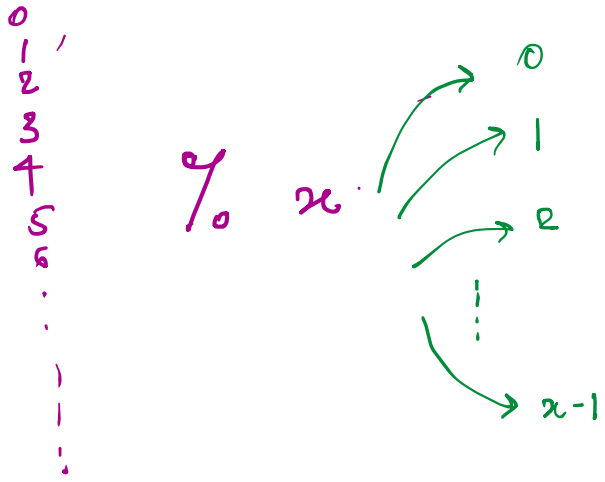
```
getShard ( ShardingKey_value) {
    ---
    ---
    ---
    ---
    ---
    ---
    return shard#.
}
```

⊛ APPROACH 1 :- MODULO

row → (sk) % (ant of Mchines)



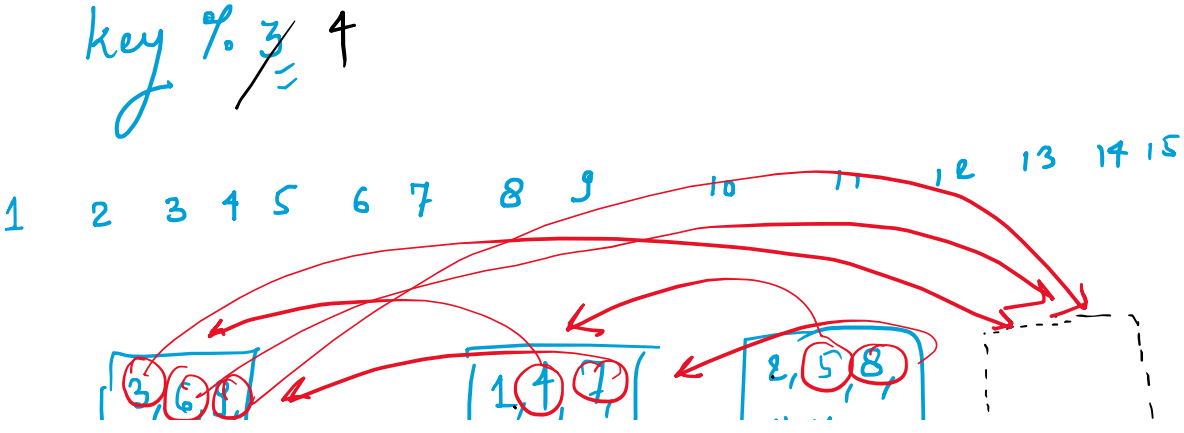
%2

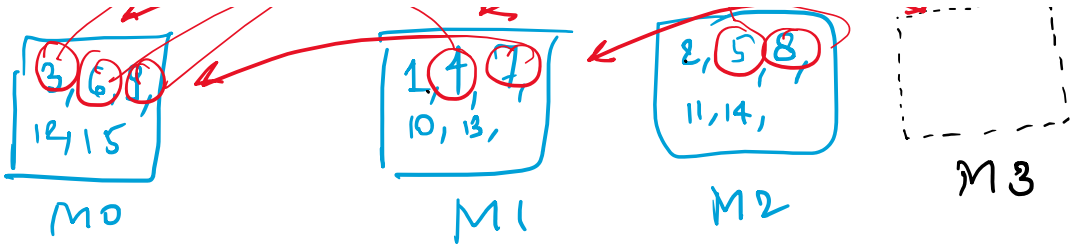


Fast ✓

Uniform dist. ✓ (Due to randomization in huge amt of data)

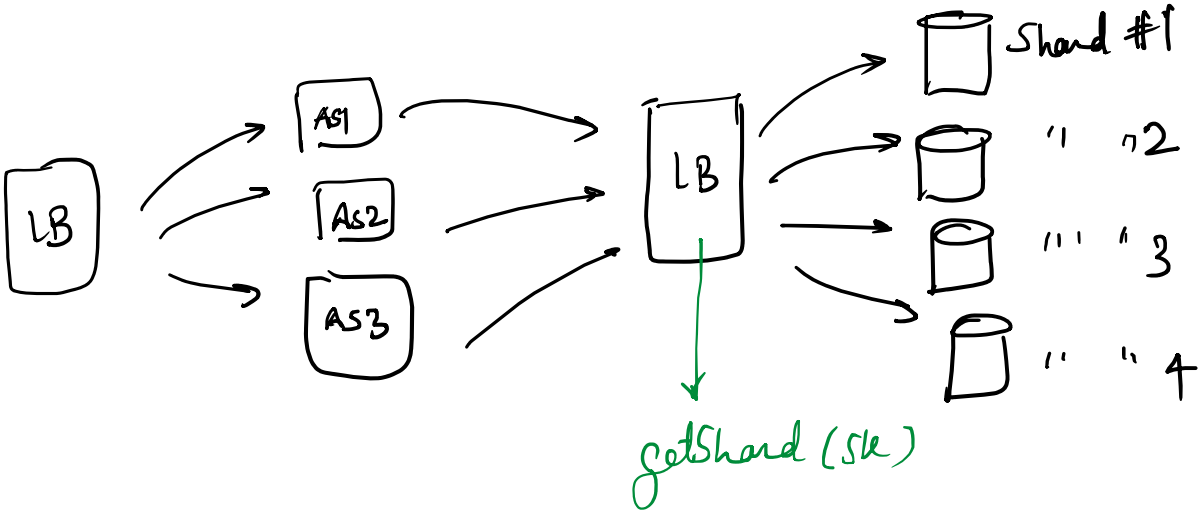
Addⁿ/Removal of shards. ✗





∴ If I'm adding a shard/ deleting a shard; a lot of data will move into/ move out of each of the shard.

leading to a lot of data movement

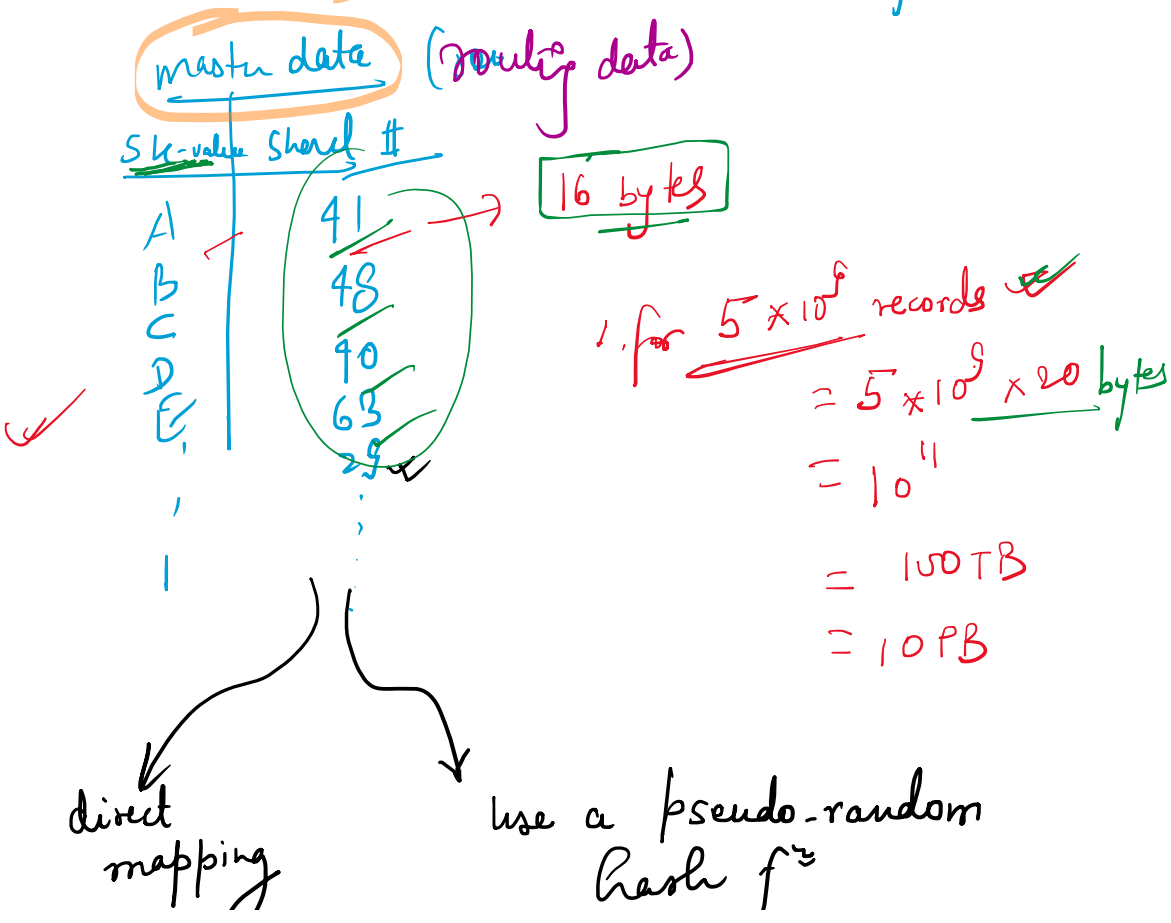


23/9/26

[1000 ids (sk values) → hash ⇓ which sk goes which shard]

⇓

this data can also bloat in future



1. for 5×10^8 records
$= 5 \times 10^8 \times 20$ bytes
$= 10^{11}$
$= 100$ TB
$= 10$ PB

mapping

use a pseudo-random hash f^n

i/p (23) → o/p random

↓ always same for same i/p

eg. SHA, MD5,

(*)

data (you want to store)

bookmarks

id	title	url	user-id (created-by)	timestamp
1	ABC	uvw	1	a → → 1
2	ABC	lmn	2	b →
3	DEF	pqr	1	c → may
4	GHI	rst	1	d
5	DEF	pqr	3	e
6	GHI	uvw	2	f

2TB (size)

↓ can't fit in 1 m/c (DB) (lets say)

↓

divide the entire data among multiple DBs

↓

Lets say this entire data of 2TB is getting divided among 5 machines (shards)

Which data goes where ?

↓

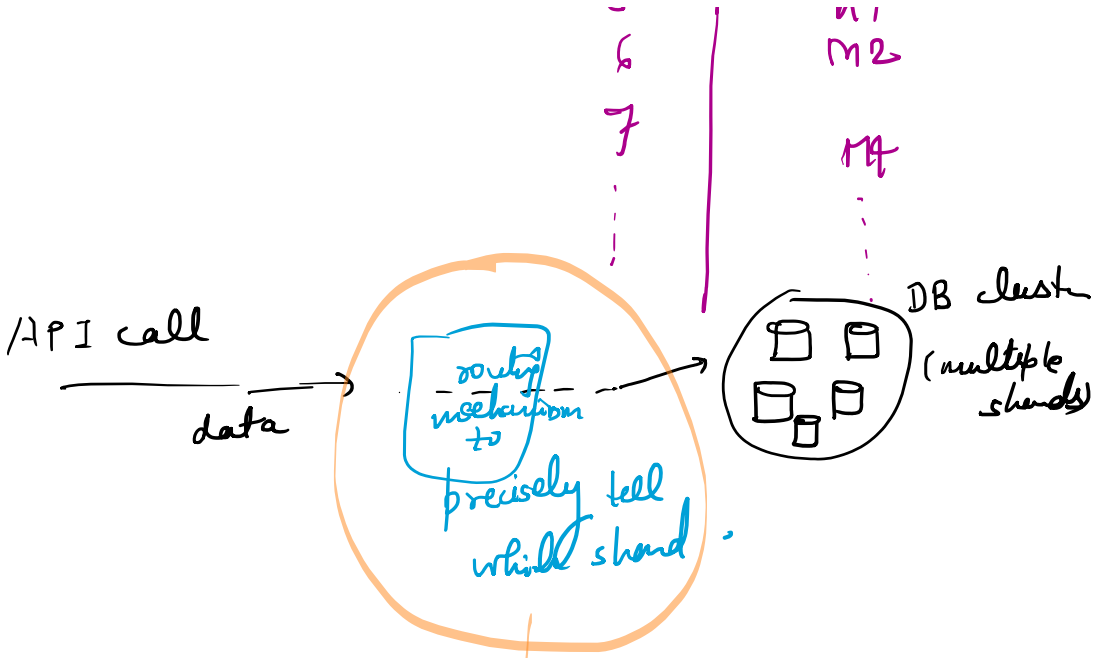
I.) 1st proposal → store direct mapping of only sk

↓

sep. table

bookmark-routing

sk (key) (user id)	Shard# (M/C #) (value)
1	M3
2	M2
3	M1
4	M5
5	M1
6	M2



getShard (sk - value)

R|W

Disadv.

the data "bookmark_routing" can itself require sharding

II.) MODULUS (already discussed)

$2^{32} - 1 \approx 4$ million

$2^{63} - 1$

$10^9 + 7$

III.) Range Based Distribution

→ set a range

→ all values in that range

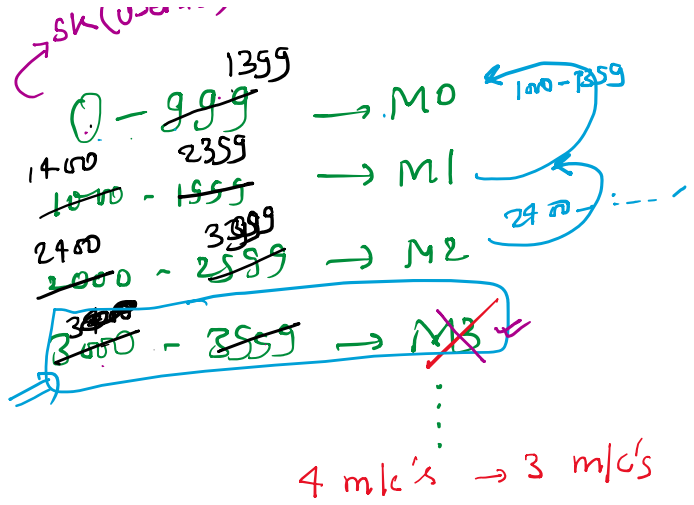
→ same m/c (shard)

SK (userId)

1359

M0 100 - 1359

Fast ✓



Fast ✓

Uniform Distribution ✗

Addⁿ/Removal of shards — more/less data movement ✓

No. of bookmarks created for/by user-id 0-999 (they joined earlier) >> that for 1000-1999.

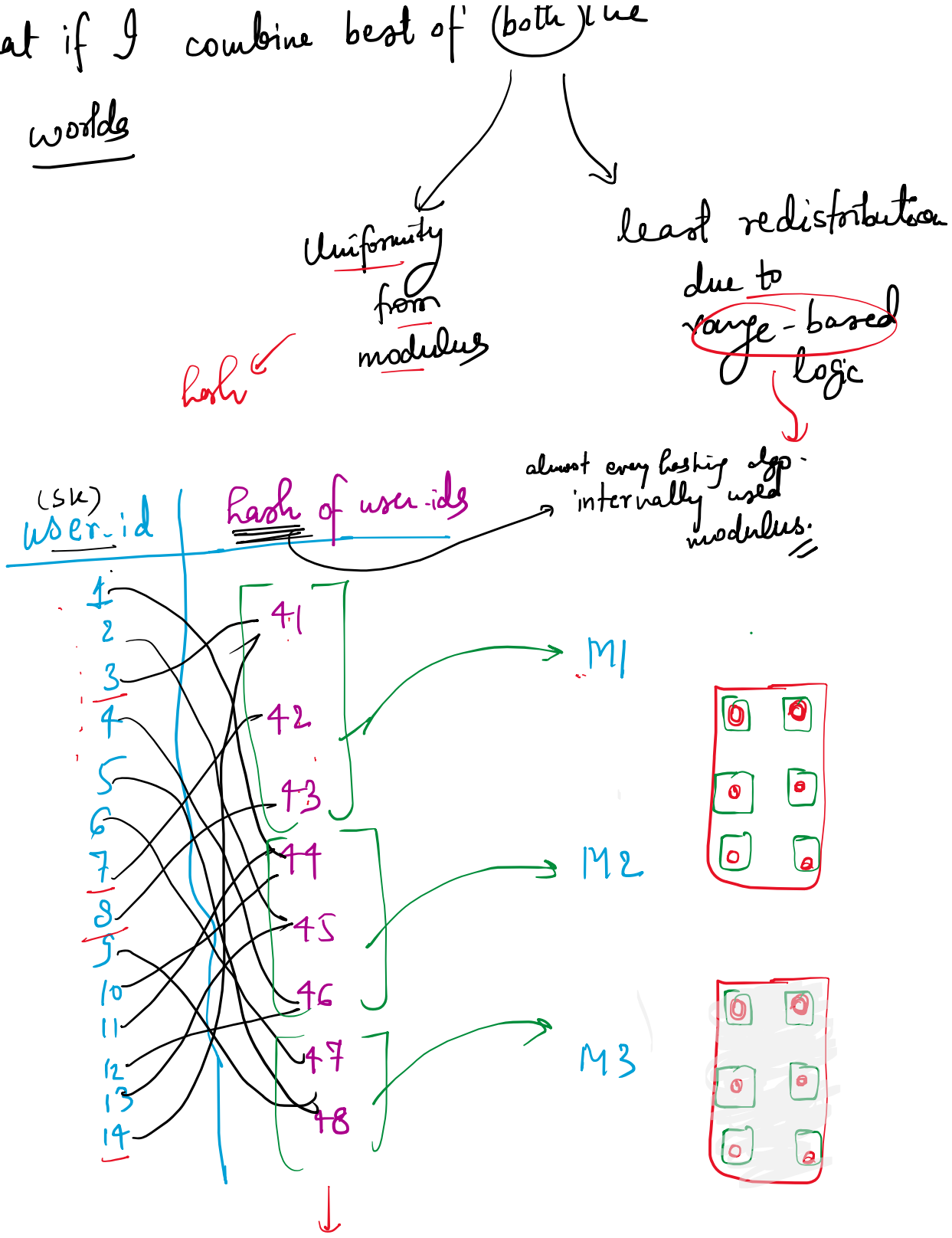
$x(-x)^2 \neq x^2$

$x = \sqrt{-1}$

$[-(-4)]^2 = 4^2$

(*) What if I combine best of both the

(*) What if I combine best of (both) the worlds



formulae OR

getShard(Sk) {

y = hash(sk)

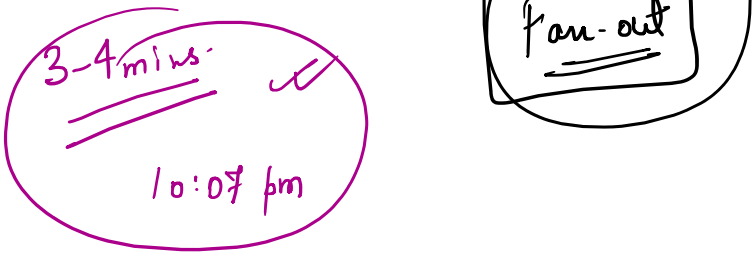
return range based distribution(y)

}

Fast ✓

Uniform Distribution ✓

Addⁿ / Deletion leading to less data ✓ Rearrangement

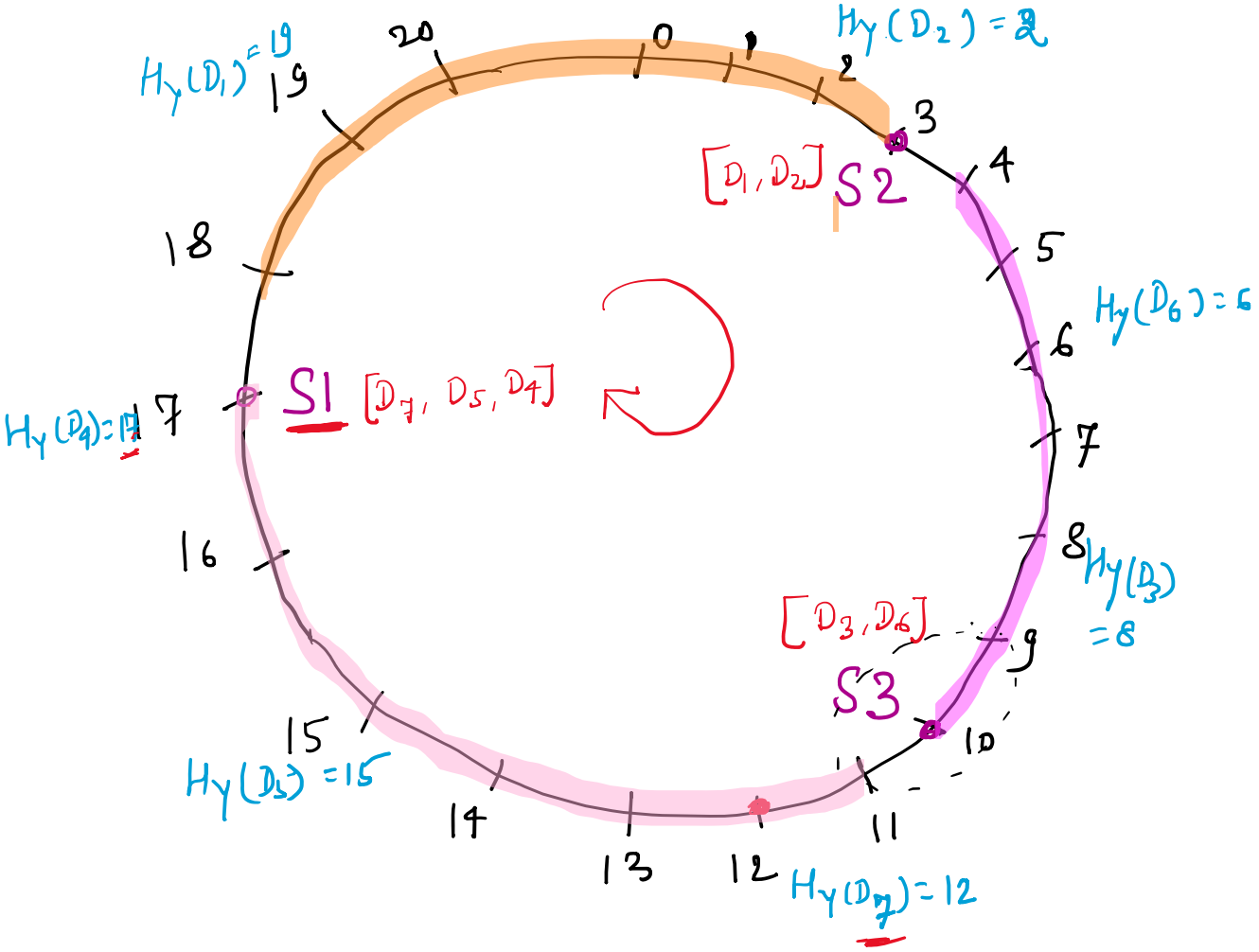


⊛ Consistent Hashing

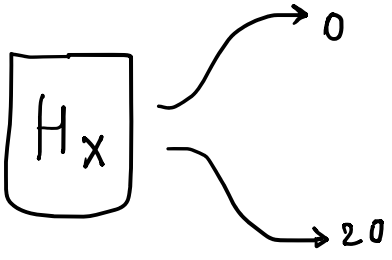
(1) Assume you've a huge virtual circular ring w/ nos. of a big enough range [In Prod:-> $0-10^6$]

big enough range [In Prod:-> 0 - 10]

↳ bring randomness.



(2.) Lets say you've a hash f^n H_x



For every server, we compute its hash value & put the server on the ring at that value.

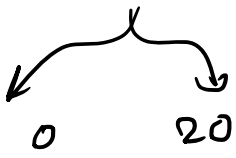
S_1 S_2 S_3

$H_X(S_1) \rightarrow 17$

$H_X(S_2) \rightarrow 3$

$H_X(S_3) \rightarrow 10$

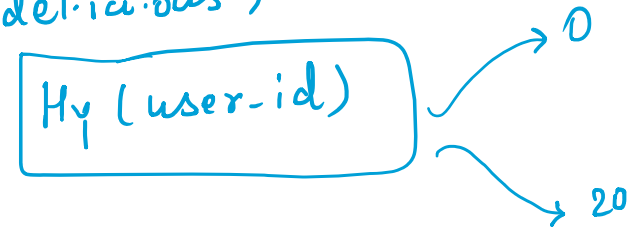
(3.) Have another hash f^{n} H_Y which also gives values in the same range



↓

For any row that you've to save, compute the hash H_Y of the Sharding key of that record

Eg: For delicious,



save the data of that row in immediate next server on the ring.

$D_1, D_2, D_3, \ldots$ ---- → SK of diff. rows

∴ Ideally

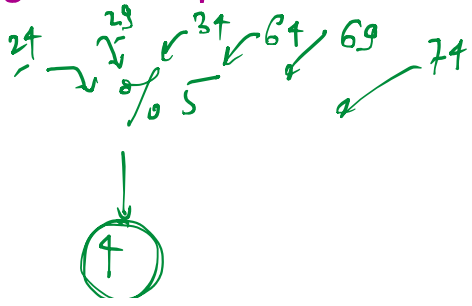
we're 1st taking hash (Hy) of SK (of the data to be stored)

& then,

doing a range-based distribution on these hashed SK-values

Q Why not H_x & H_y ——— be same single hash-fn

Ans: To avoid collisions.

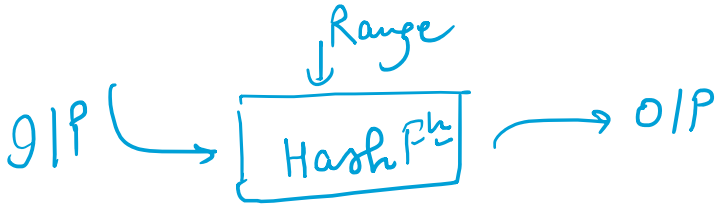


why to avoid it?

4

to bring uniform distribution by bringing randomness

26/9/26



H_x

⇓

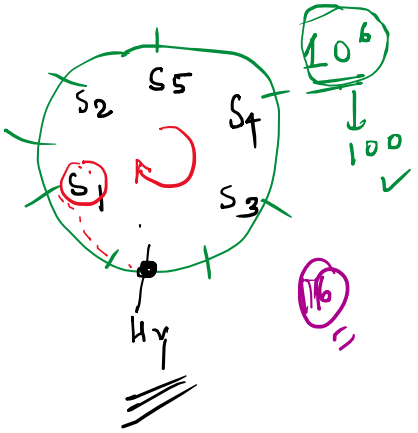
Server

H_y

⇓

Sharding key (user_id)

* Fast



$\sim 10^5$

$\sim 10^3$ to 10^4

key serves not ordered; ↑ loc^n value

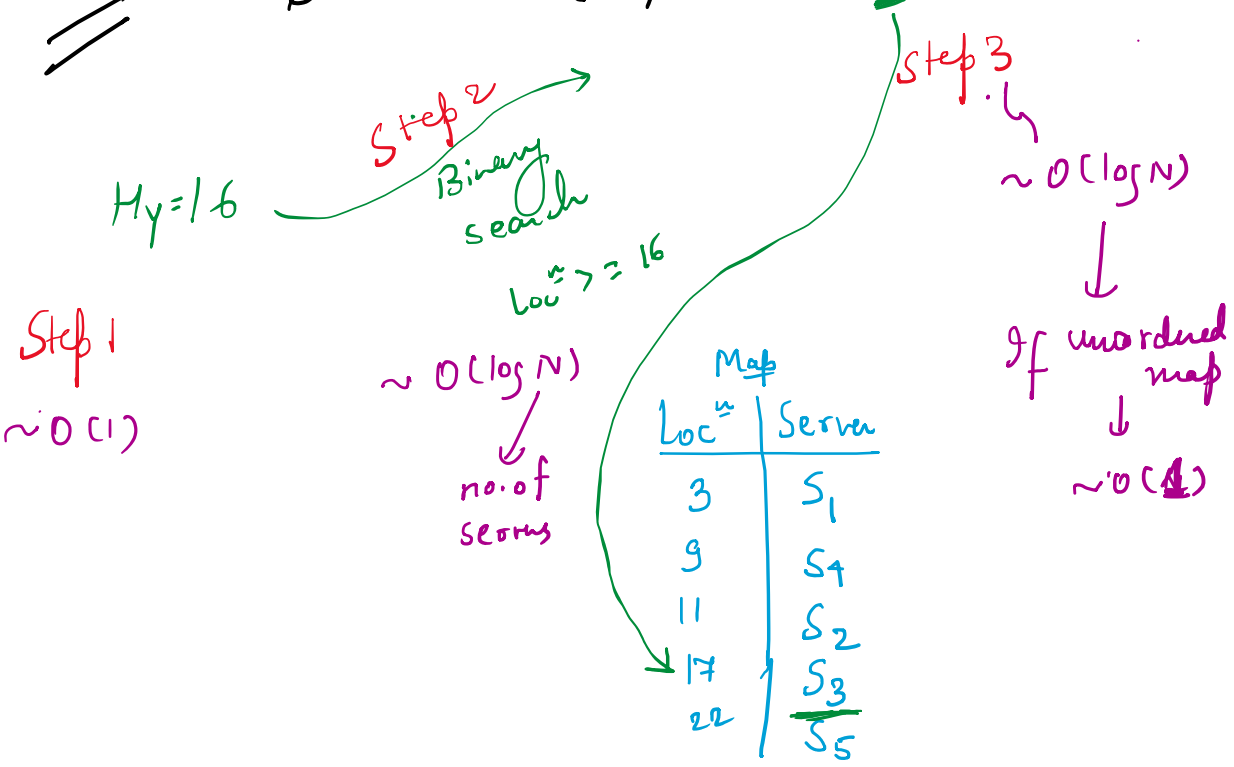
Alt 1

(16)

key: servers	value: loc^n
S_1	3
S_2	11
S_3	17
S_4	9
S_5	22

Sol^n

Server loc^n = [3, 9, 11, 17, 22]



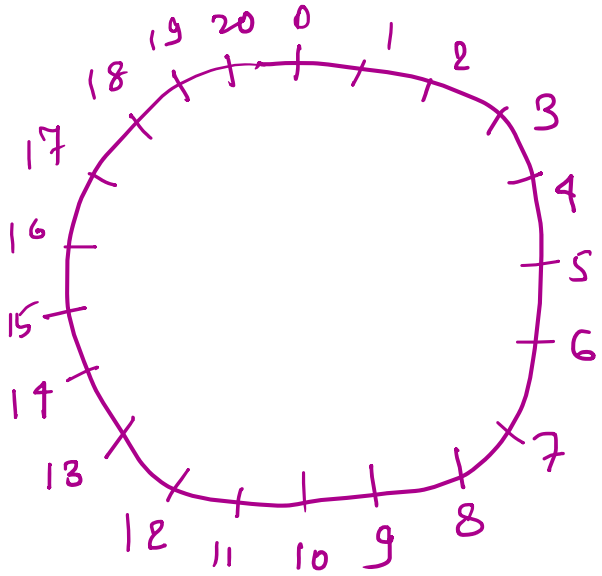
Overall → ~log N , N = no. of servers.

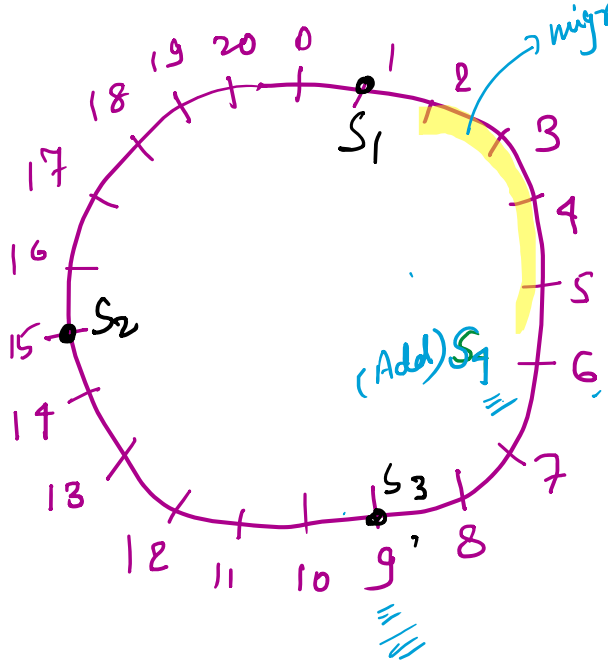
→ Uniform Distribution ✓

Because → Range in virtual ring is huge
→ Hashing

↳ Hashing :/

→ Addⁿ of New Servers





O(N) → O(1)
Doubly Linked list

loc^n → [1 | 4 | 9 | 15]

Map →

	Server
1	
9	0.
15	

6. S_4 (log N)

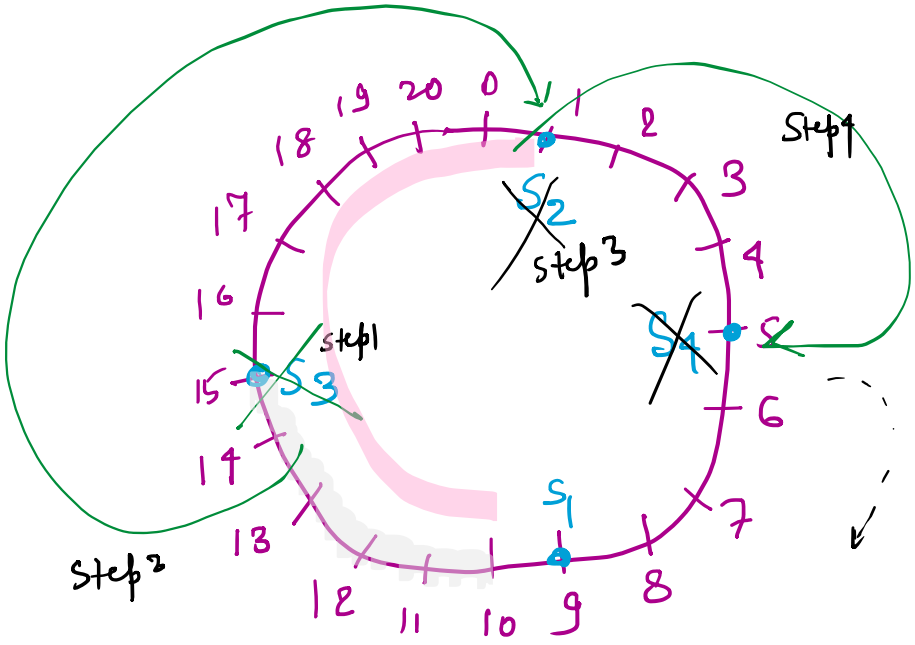
≈ log(N)

Overall, one time effort

BUT

feasible =

(A) Deletion of a shard



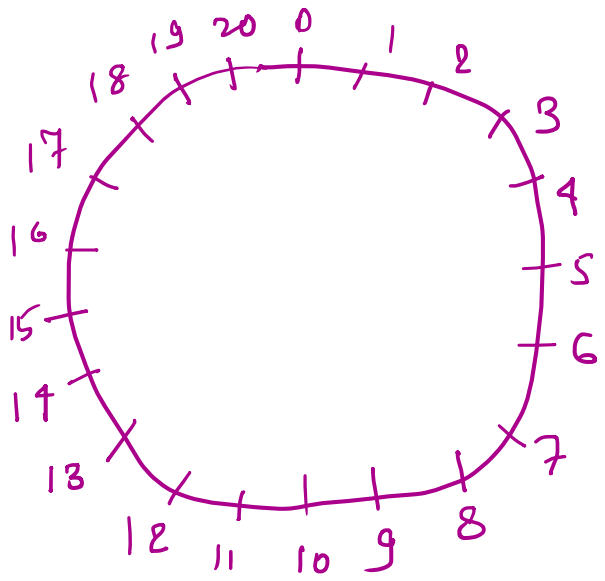
Most obvious

cause of S_3 failure → overload. (crash)

"Cascading Failure" → Prob

↓ Sol^n

Optimized Consistent Hashing



Earlier — H_x(server_id)

for each server, we were finding its hash value & putting on server ring

Now — Instead of having 1 hash f^{n} for servers, we'll have multiple hash f^{n} of servers

H_{x1} (server_id)

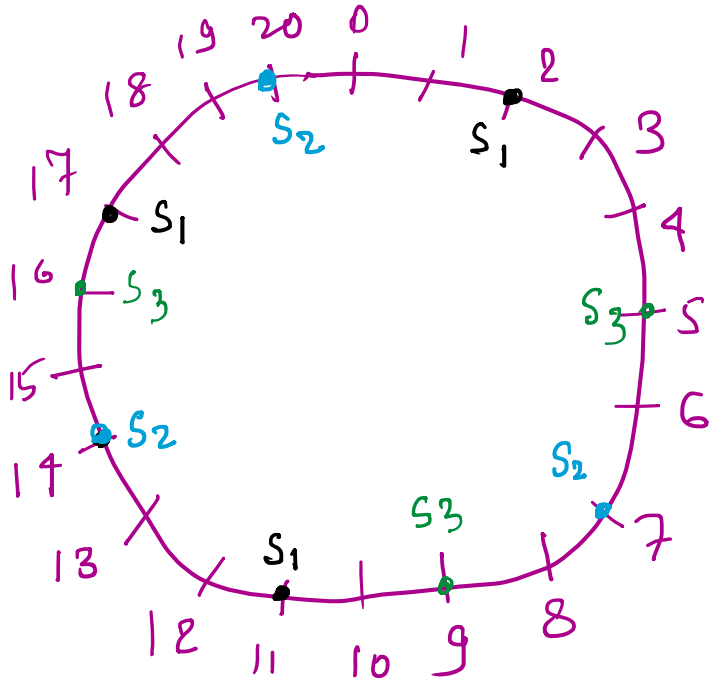
H_{x2} (")

H_{x3} (")

for every server, I will put it on 3 places in my

Q on 3 places in my hash ring.

- Rest overall further process remains same =>



$S_1 \rightarrow$ $H_{x1}(S_1) = 11$

$H_{x2}(S_1) = 2$

$H_{x3}(S_1) = 17$

$S_2 \rightarrow H_{x1}(S_2) = 14$

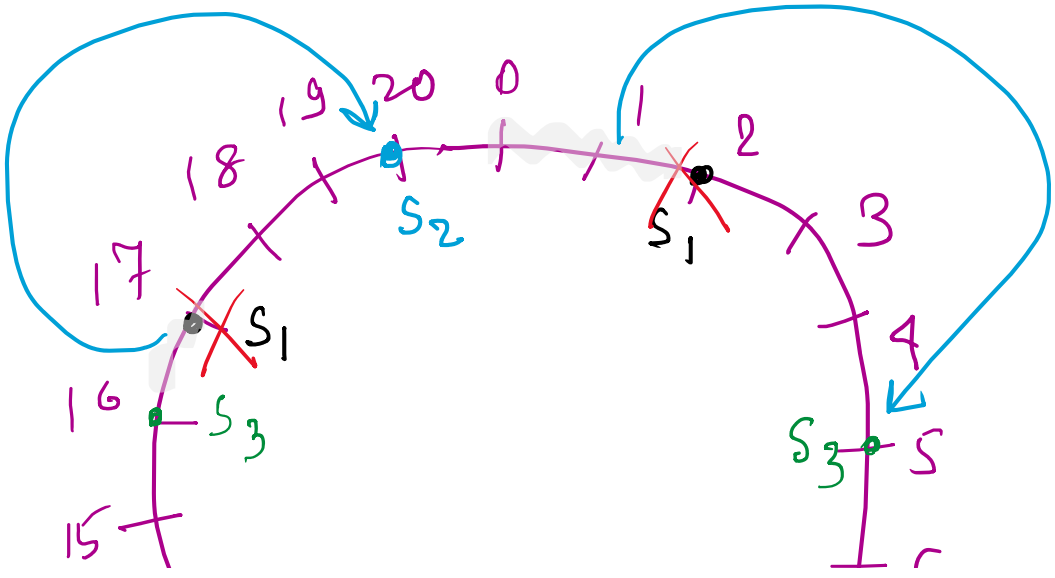
$H_{x2}(S_2) = 7$

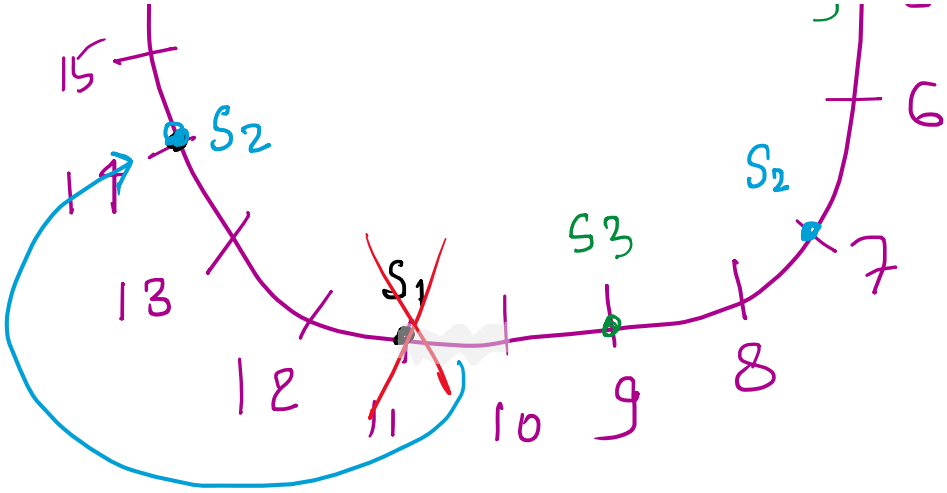
$H_{x3}(S_2) = 19$

$S_3 \rightarrow H_{x1}(S_3) = 9$

$H_{x2}(S_3) = 16$

$H_{x3}(S_3) = 5$





Note :- There are no replicas / multiple copies of a server

(Virtual Ring)

↓

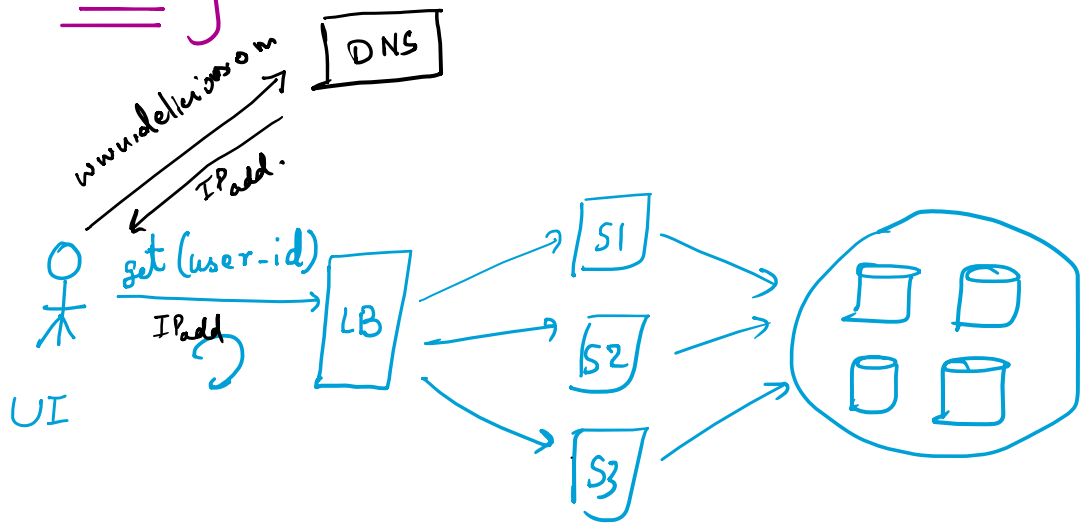
now, distributed range is being handled by each server

↓

∴ Now, If a server dies, same data will be distributed among multiple servers.

1:09 pm

Caching



If I am going every 1 min to DB, probably I'm doing useless work

If I'm fetching IP add. of del.icious everytime, I am doing useless work.

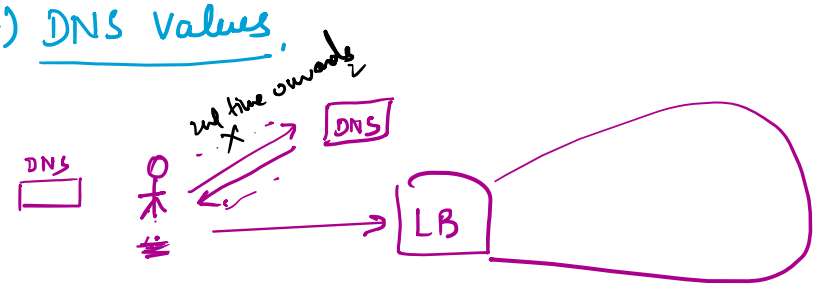
Caching

Storing/Precomputing data @ some place so as to not fetch it again & again from orig. source & thus to overall improve the performance of any appⁿ

⊛ Types of Caching.

1.) In Browser Caching.

a.) DNS Values.

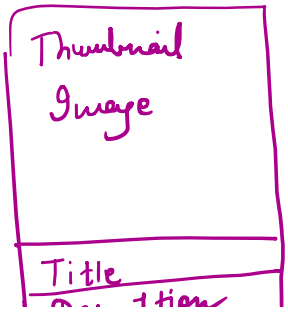


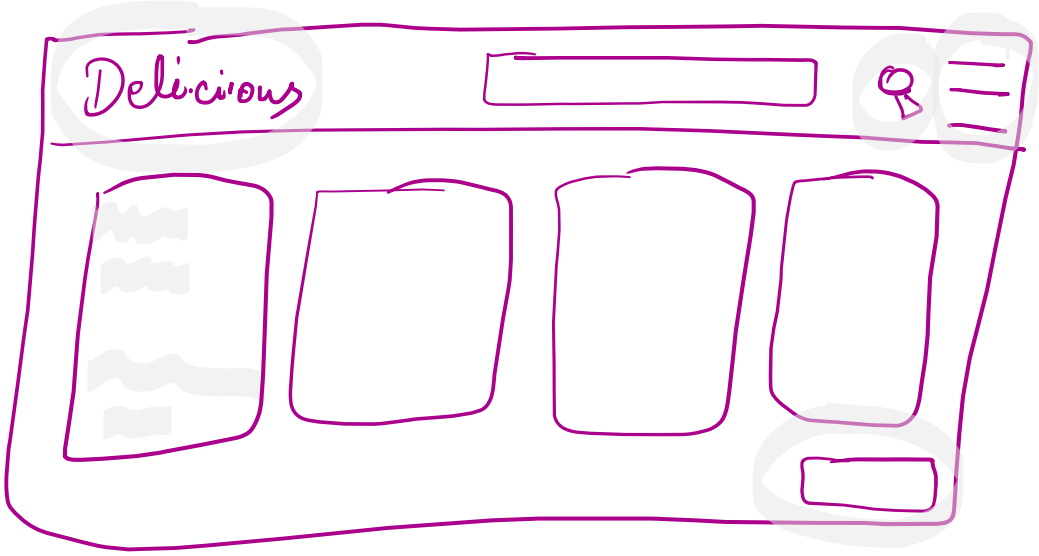
Store the DNS value locally so that subsequent fetch doesn't go to DNS

local Storage

lite-weight storage @ browser level

b.) Images (static files/images/ CSS - styling etc)





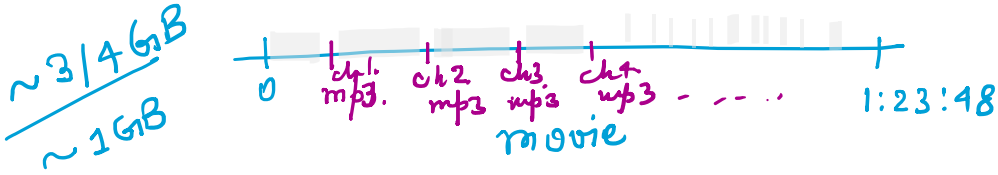
store images / styling etc.
also on the local storage

2.) CDNs

↳ Content Delivery N/w.

- Live Streaming
- Stream videos from
  - ↳ YT
  - ↳ A.Prime

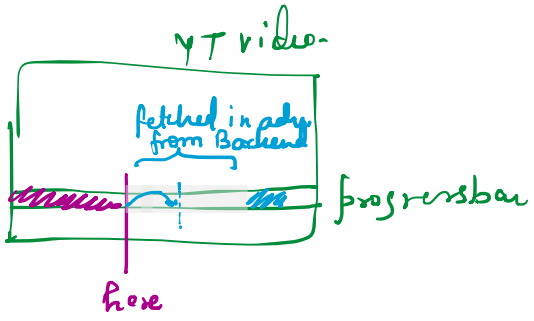
↳ Netflix ?



↓

can't send complete data @ once.

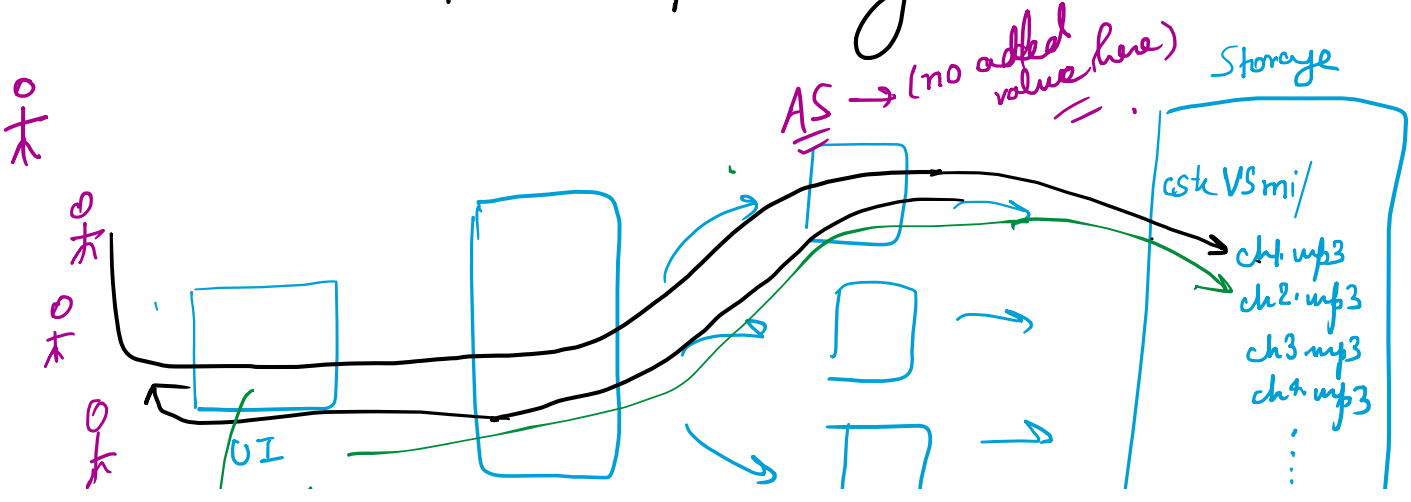
↳ divide the video into chunks.

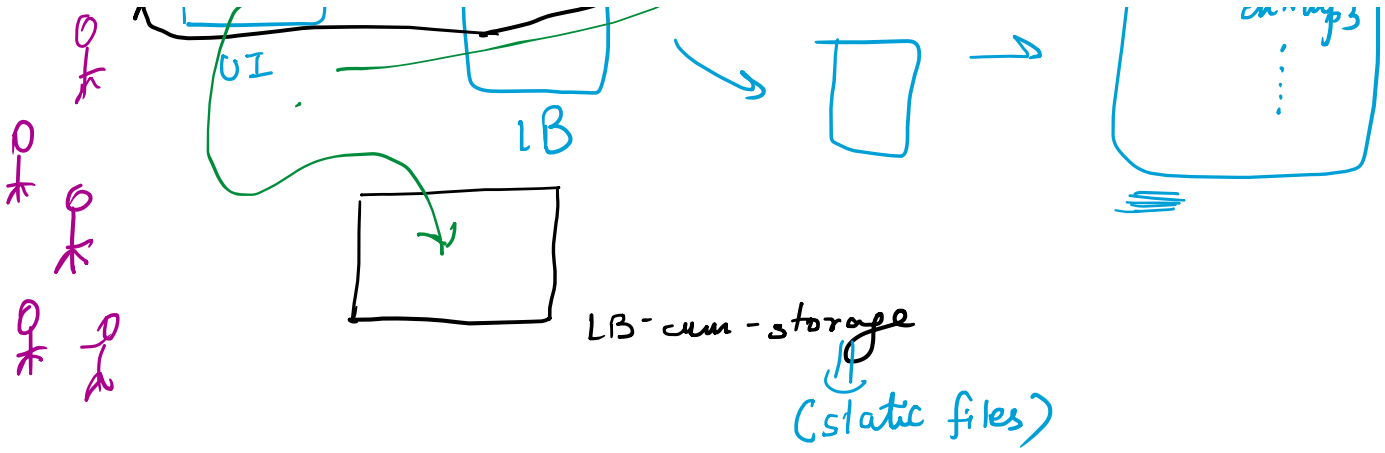


Live Streaming

↳ shoot the video.

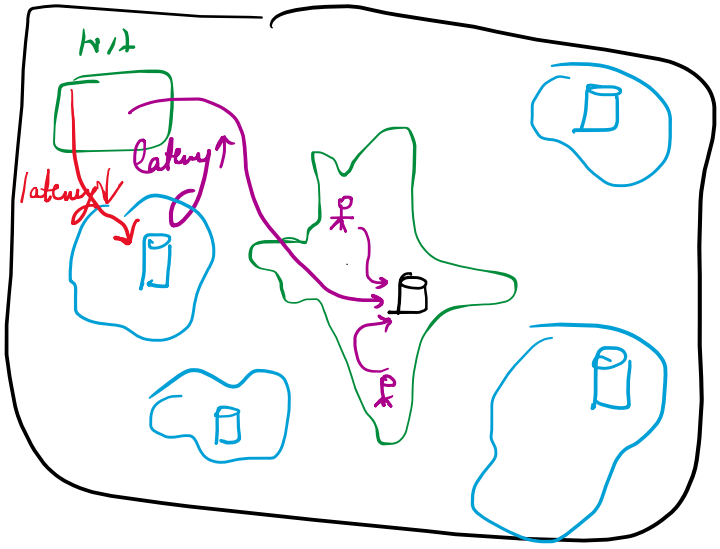
& send it periodically





CDNs

Storage servers that are distributed across the globe.



CDN providers

- Akamai — JioHotstar
- Cloudfront — AWS
- Cloudfare — Google Cloud
- Fastly